

CLÍNICA ODONTOLÓGICA

Sistema de Gestión de Pacientes, Profesionales y Turnos

Consigna del Trabajo Final — Especificación de Requerimientos

1. Objetivo

Desarrollar una aplicación de escritorio que permita a una clínica odontológica administrar de forma integral su padrón de **pacientes**, su plantel de **odontólogos** (con especialidades diferenciadas), su personal de **secretaría** y la **agenda de turnos** que vincula a los tres, incluyendo el cálculo del monto a facturar por cada atención y la persistencia de todos los datos entre ejecuciones.

El trabajo debe resolverse aplicando el paradigma orientado a objetos: encapsulamiento, herencia, polimorfismo, abstracción, uso de interfaces genéricas, manejo de excepciones propias del dominio y una arquitectura en capas con responsabilidades separadas.

2. Alcance

- **Incluido:** ABM completo de pacientes, odontólogos, secretarias y turnos; validaciones de negocio; asignación de turnos con control de disponibilidad y de compatibilidad profesional; consultas y filtros; facturación por turno; persistencia en archivos; interfaz gráfica de escritorio.
- **Excluido:** autenticación y perfiles de usuario, historia clínica odontológica (odontograma, tratamientos, prescripciones), facturación fiscal o integración con obras sociales, notificaciones por correo o SMS, y acceso multiusuario concurrente.

3. Modelo de Dominio

El sistema se organiza en torno a una jerarquía de personas y a la entidad Turno, que actúa como nexo del negocio.

3.1 Entidades y atributos

Entidad	Atributos	Observaciones
Persona (abstracta)	id (Long), nombre (String), apellido (String), dni (Integer)	Clase base de la jerarquía. Concentra los datos comunes a toda persona del sistema.
Paciente <i>extends Persona</i>	email (String), fechaAlta (LocalDate), domicilio (Domicilio), obraSocial (Boolean), historialPaciente (List<Turno>)	La fecha de alta se asigna automáticamente al registrarlo. Debe poder ordenarse de forma natural por apellido y luego por nombre.
Domicilio	calle (String), numero (Integer), localidad (String), provincia (String)	Clase de soporte del paciente. No se administra de forma autónoma: no existe fuera del paciente que la contiene.
Odontólogo (abstracta, <i>extends Persona</i>)	matricula (String), historialOdontologo (List<Turno>)	Define tres operaciones abstractas que cada especialidad resuelve a su manera: especialidad, tarifa base y qué motivos de consulta puede atender.
Secretaria <i>extends Persona</i>	historialSecretaria (List<Turno>)	Registra qué turnos gestionó cada secretaria.
Turno	id (Long), paciente, odontologo, secretaria, fecha (LocalDate), hora (LocalTime), motivoConsulta (String), estado (EstadoTurno)	Nace siempre en estado PENDIENTE. Debe poder generar un mensaje de recordatorio legible con sus datos.
EstadoTurno (enumerado)	PENDIENTE, CONFIRMADO, CANCELADO, COMPLETADO	Modela el ciclo de vida del turno.

3.2 Especialidades odontológicas

Se requieren al menos tres especializaciones concretas de Odontólogo. Cada una define su nombre de especialidad, su tarifa base y el conjunto de motivos de consulta que está habilitada a atender:

Subclase	Especialidad	Tarifa base	Motivos de consulta que atiende
OdontologoGeneral	Odontología General	\$ 50.000	consulta, control, limpieza, caries, dolor
Ortodoncista	Ortodoncia	\$ 70.000	ortodoncia, brackets, alineadores, mordida, apiñamiento
Endodoncista	Endodoncia	\$ 80.000	endodoncia, conducto, pulpa, infección, dolor intenso

La verificación de compatibilidad entre el motivo de consulta y el profesional debe resolverse **polimórficamente**: la capa de servicios pregunta al odontólogo si puede atender el motivo, sin conocer ni preguntar por su tipo concreto (sin instanceof ni cadenas de if por especialidad).

3.3 Relaciones

Relación	Tipo	Cardinalidad	Descripción
Persona → Paciente / Odontólogo / Secretaria	Herencia	—	Generalización de los datos comunes a toda persona.
Odontólogo → General / Ortodoncista / Endodoncista	Herencia	—	Especialización con comportamiento propio de cada especialidad.
Paciente → Domicilio	Composición	1 : 1	Todo paciente tiene un domicilio; el domicilio no existe sin el paciente.
Turno → Paciente	Asociación	N : 1	Un turno pertenece a un paciente; un paciente puede tener muchos turnos.
Turno → Odontólogo	Asociación	N : 1	Un turno es atendido por un odontólogo; un odontólogo tiene muchos turnos.
Turno → Secretaria	Asociación	N : 1	Un turno es gestionado por una secretaria; una secretaria gestiona muchos turnos.
Paciente / Odontólogo / Secretaria → Turno	Asociación (historial)	1 : N	Cada persona mantiene la lista de sus turnos. La relación es bidireccional y debe sincronizarse en cada alta y baja de turno.

4. Requerimientos Funcionales

RF1 — Gestión de Pacientes

- Registrar un paciente con sus datos personales, email, indicación de obra social y su domicilio completo.
- Buscar paciente por ID y por DNI.
- Listar todos los pacientes y listarlos ordenados alfabéticamente por apellido y nombre.
- Modificar los datos del paciente, incluido su domicilio.
- Eliminar un paciente, siempre que no tenga turnos futuros activos.

RF2 — Gestión de Odontólogos

- Registrar un odontólogo indicando su especialidad, que determina la subclase concreta a instanciar.
- Buscar odontólogo por ID y por matrícula.
- Listar todos los odontólogos, mostrando su especialidad.
- Modificar los datos del odontólogo.
- Eliminar un odontólogo, siempre que no tenga turnos futuros activos.

RF3 — Gestión de Secretarías

- Registrar, buscar por ID y por DNI, listar, modificar y eliminar secretarías.

RF4 — Gestión de Turnos

- **Reservar turno:** vincular un paciente, un odontólogo y una secretaria en una fecha y hora determinadas, indicando el motivo de consulta. El turno se crea en estado PENDIENTE.
- **Modificar turno:** permitir cambiar odontólogo, secretaria, fecha, hora, motivo y estado, revalidando todas las reglas de negocio.
- **Cambiar estado:** confirmar, cancelar o completar un turno.
- **Eliminar turno,** quitándolo de los historiales de las tres personas involucradas.
- **Buscar turno por ID** y listar todos los turnos.

RF5 — Consultas y Filtros

El sistema debe ofrecer las siguientes búsquedas sobre la agenda, con los resultados ordenados por fecha y hora:

- Turnos de un paciente determinado.
- Turnos de un odontólogo determinado.
- Turnos gestionados por una secretaria determinada.
- Turnos por estado (pendientes, confirmados, cancelados, completados).
- Turnos comprendidos en un rango de fechas.
- Turnos de un odontólogo para un paciente en particular.

RF6 — Facturación del Turno

El sistema debe calcular el monto a abonar por un turno mediante un componente dedicado al cálculo:

- Si el paciente **tiene obra social**, abona un copago fijo de \$ **10.000**, independientemente de la especialidad.
- Si el paciente **no tiene obra social**, abona la **tarifa base del odontólogo** que lo atiende, según su especialidad.

RF7 — Persistencia de Datos

- Los datos de pacientes, odontólogos, secretarías y turnos deben sobrevivir entre ejecuciones, almacenados en **archivos de texto plano delimitados**, uno por entidad, dentro de una carpeta de datos del proyecto.
- La carga debe realizarse automáticamente al iniciar la aplicación; el guardado, al cerrarla, previa confirmación del usuario.
- Al reconstruir los turnos deben resolverse las referencias a paciente, odontólogo y secretaria a partir de sus identificadores, y deben restaurarse los historiales de cada persona.
- Los contadores de identificadores deben reanudarse a partir del máximo valor cargado, para evitar colisiones de IDs.
- La primera ejecución, sin archivos previos, debe funcionar normalmente partiendo de repositorios vacíos.

RF8 — Interfaz Gráfica

- Ventana principal con navegación por **pestañas**: Pacientes, Odontólogos, Secretarías y Turnos.
- Cada pestaña debe combinar un **formulario** de alta/edición, una **tabla** de listado y botones de acción (nuevo, guardar, eliminar, buscar, limpiar). La selección de una fila debe cargar el registro en el formulario.
- La pestaña de turnos debe permitir seleccionar paciente, odontólogo y secretaria desde listas desplegables, elegir el estado, calcular el monto del turno y aplicar los filtros del RF5.
- Las listas desplegables deben mantenerse actualizadas frente a los cambios realizados en las otras pestañas, sin necesidad de reiniciar la aplicación.
- Todos los errores de negocio deben mostrarse al usuario en cuadros de diálogo, con un mensaje claro. Ninguna excepción debe llegar sin control a la consola ni interrumpir la aplicación.
- Al cerrar la ventana debe ofrecerse guardar los datos, con opción de cancelar el cierre.

5. Reglas de Negocio y Validaciones

Ámbito	Regla
Datos personales	Nombre y apellido obligatorios y sin dígitos numéricos. DNI obligatorio y positivo.
Unicidad	No pueden existir dos pacientes con el mismo DNI, dos secretarías con el mismo DNI, ni dos odontólogos con la misma matrícula. La validación también rige al modificar un registro existente.
Paciente	Email obligatorio. La condición de obra social debe estar definida (sí / no). El domicilio es obligatorio y completo: calle, número positivo, localidad y provincia.
Odontólogo	Matrícula obligatoria.
Turno — existencia	El paciente, el odontólogo y la secretaria del turno deben existir previamente en el sistema.
Turno — motivo	El motivo de consulta es obligatorio y debe ser compatible con la especialidad del odontólogo asignado.
Turno — disponibilidad	Un odontólogo no puede tener dos turnos en la misma fecha y hora. Al modificar un turno, la validación debe excluir al propio turno de la comparación.
Turno — datos	Fecha, hora y estado son obligatorios.
Bajas	No se puede eliminar un paciente ni un odontólogo que tenga turnos con fecha igual o posterior al día actual en estado PENDIENTE o CONFIRMADO. Debe informarse el motivo del rechazo.
Consistencia	Toda alta, baja o reasignación de un turno debe reflejarse en los historiales del paciente, el odontólogo y la secretaria involucrados.

6. Requerimientos No Funcionales

ID	Requerimiento	Descripción
RNF1	Tecnología	Utilizar Java SE puro, sin frameworks externos ni dependencias de terceros: colecciones (<code>Map</code> , <code>List</code>), <code>Stream</code> y <code>Comparator</code> para búsquedas, filtros y ordenamientos, <code>java.time</code> para fechas y horas, <code>java.io</code> para archivos y Swing para la interfaz gráfica.
RNF2	Arquitectura en capas	Separar el sistema en entidades, repositorios, servicios, controladores, vista, persistencia y excepciones. Cada capa se comunica únicamente con las adyacentes; la vista nunca accede directamente a los repositorios.
RNF3	Abstracciones genéricas	Definir una interfaz genérica de repositorio con las operaciones CRUD y una interfaz genérica de servicio, e implementarlas para cada entidad, de modo que agregar una nueva entidad no obligue a modificar la arquitectura existente.
RNF4	Excepciones propias	Diseñar una jerarquía de excepciones del dominio con una excepción base común y subclases específicas para dato inválido, paciente inexistente, odontólogo inexistente y turno ya reservado. Toda regla de negocio incumplida debe comunicarse mediante estas excepciones, no con valores nulos ni códigos de retorno.
RNF5	Robustez	Ningún error debe provocar la caída de la aplicación. La capa de presentación captura las excepciones del dominio y las traduce en mensajes comprensibles para el usuario.
RNF6	Calidad de código	Convenciones estándar de Java (<code>PascalCase</code> para clases, <code>camelCase</code> para métodos y variables, constantes en mayúsculas), nombres significativos, métodos cortos con una única responsabilidad, sin código duplicado ni código muerto.
RNF7	Diseño OO	Encapsulamiento estricto (atributos privados, acceso mediante métodos), preferencia por la composición cuando no exista una relación "es-un" genuina, programación contra interfaces, alta cohesión y bajo acoplamiento.
RNF8	Versionado	Utilizar Git con commits frecuentes y descriptivos, y un archivo <code>.gitignore</code> que excluya binarios y archivos de configuración del IDE.

7. Arquitectura Esperada

entity/	Modelo de dominio: Persona, Paciente, Domicilio, Odontologo y sus especialidades, Secretaria, Turno, EstadoTurno.
repository/	Almacenamiento en memoria sobre colecciones, detrás de una interfaz genérica de repositorio. Búsquedas por clave (DNI, matrícula) y detección de conflictos de horario.
service/	Lógica de negocio: validaciones, reglas, sincronización de historiales y componente de facturación. Lanza las excepciones del dominio.
controller/	Intermediario entre la vista y los servicios. Traduce los datos del formulario en objetos del dominio y delega la operación.
view/	Interfaz gráfica Swing: ventana principal con pestañas y un panel por entidad.
persistence/	Lectura y escritura de los archivos de datos, resolución de referencias entre entidades y restauración de contadores de ID.
exception/	Jerarquía de excepciones propias del dominio.

8. Entregables

- **Código fuente** completo del proyecto Java, compilable y ejecutable sin errores.
- **Ejecutable** del sistema (archivo JAR).
- **Diagramas UML** de clases del sistema, reflejando la jerarquía de herencia, las interfaces, las relaciones entre entidades con sus cardinalidades y la organización en capas.
- **Informe técnico** que documente la arquitectura, las decisiones de diseño adoptadas y su justificación, los conceptos de POO aplicados y una reflexión final sobre el desarrollo.
- **Manual de usuario** con capturas de la interfaz y la descripción de cada funcionalidad.

9. Criterios de Evaluación

Criterio	Peso	Qué se evalúa
Funcionalidad	40 %	Todas las operaciones requeridas funcionan correctamente, con los datos persistiendo entre ejecuciones y sin errores en tiempo de ejecución.
Diseño orientado a objetos	30 %	Encapsulamiento, herencia y polimorfismo bien aplicados; uso efectivo de interfaces y abstracciones; relaciones correctamente modeladas; separación real de responsabilidades entre capas.
Calidad de código	20 %	Código limpio, legible y consistente, sin duplicación, con nombres significativos y manejo de errores coherente.
Documentación	10 %	Diagramas UML coherentes con el código entregado e informe con reflexión técnica sobre las decisiones tomadas.